Parte III · Escalabilidad

Caché, CDN, particionamiento, balanceo, microservicios y mensajería

Sistemas Distribuidos

Basado en R. Vitillo, *Understanding Distributed Systems*, capítulos 14 a 23

Universidad Industrial de Santander

Agenda

1. **Patrones de escalabilidad**
2. **Acercar los datos al cliente** — caché HTTP (cap. 14) y CDN (cap. 15)
3. **Particionar y almacenar** — particionamiento (cap. 16), archivos (cap. 17), datos (cap. 19)
4. **Repartir la carga** — balanceo (cap. 18) y caché (cap. 20)
5. **Descomponer** — microservicios (cap. 21), planos de control y datos (cap. 22), mensajería (cap. 23)

¿Qué significa escalar?

- Un sistema es **escalable** si atiende **más carga** agregando **recursos**, sin rediseñarse
- Carga: peticiones por segundo, usuarios concurrentes, volumen de datos…
- **Vertical** (*scale up*): máquina más grande → simple, pero con **techo** y costo creciente
- **Horizontal** (*scale out*): **más máquinas** → sin techo teórico, pero exige **coordinación**

> La Parte III trata del escalado **horizontal**, apoyándose en lo visto en las Partes I y II.

Tres patrones



Escalar horizontalmente: repartir el trabajo entre muchas máquinas

Cómo crece una aplicación (estilo *Cruder*)

Problema	Patrón	Técnica	Cap.
Los estáticos saturan el servidor	Replicación	Caché HTTP, CDN	14–15
Un servidor no basta	Replicación	Réplicas sin estado + balanceador	18
La base de datos no da abasto en lecturas	Replicación	Réplicas de lectura, caché	19–20
Los datos no caben en un nodo	Particionamiento	*Sharding*, NoSQL, *blobs*	16, 17, 19
El código y los equipos crecen	Descomposición	Microservicios, *gateway*, mensajería	21–23

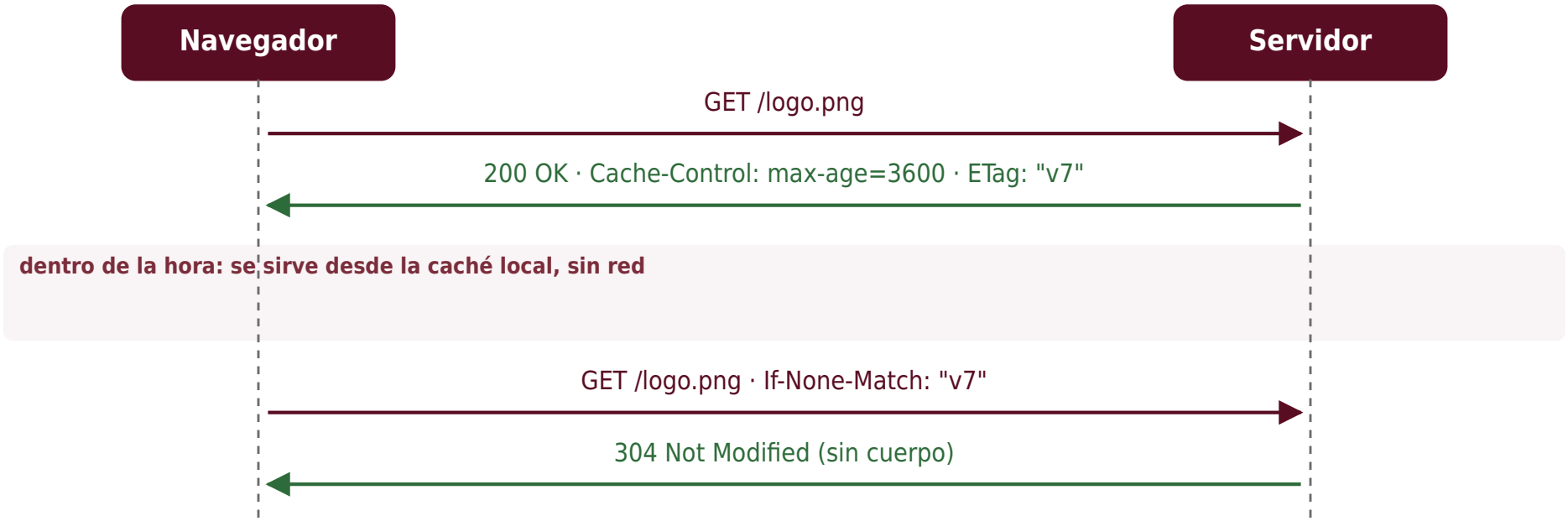
1. Acercar los datos al cliente

Caché HTTP · CDN (caps. 14–15)

Caché HTTP (cap. 14)

- Recursos **estáticos** (imágenes, CSS, JS) cambian poco y se piden mucho
- HTTP permite que el **cliente** los guarde y reutilice
- El servidor controla la vigencia con `Cache-Control: max-age=<segundos>`
- Al vencer, la copia queda **obsoleta** (*stale*) y se **revalida** con el servidor
- La caché HTTP es **eventualmente consistente**

Expiración y revalidación

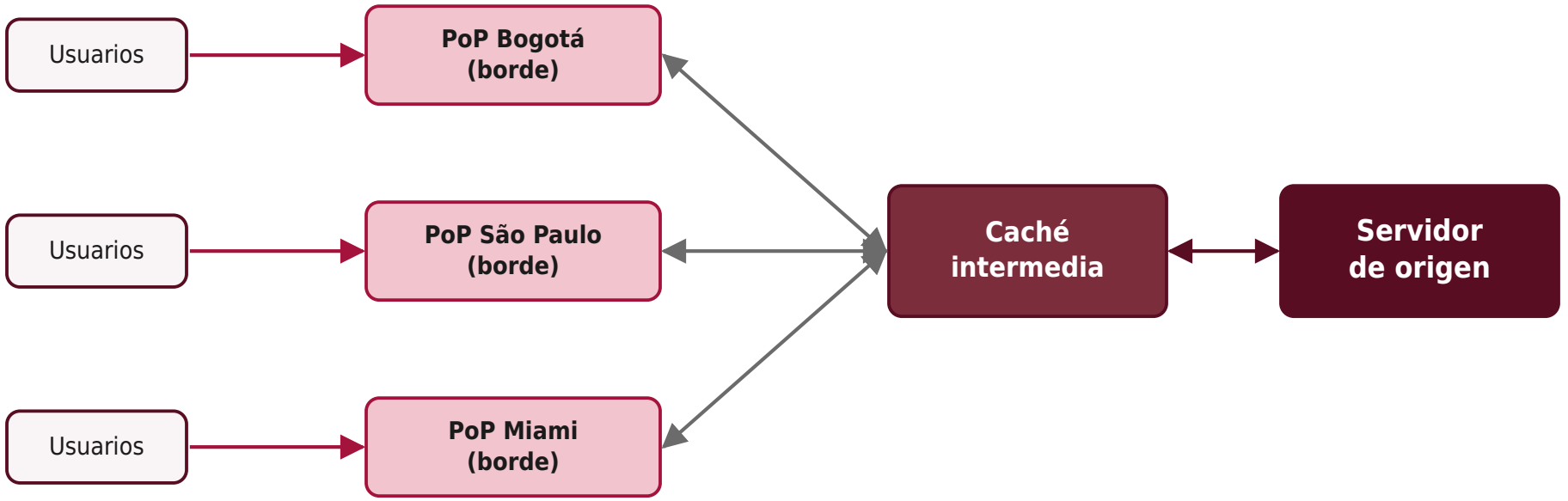


Versionar URLs y proxies inversos

```
GET /static/app.3f9a2c.js
200 OK
Cache-Control: public, max-age=31536000, immutable
```

- Un `max-age` largo impide ver cambios → **cambiar la URL** en cada versión (hash del contenido)
- **Proxy inverso**: servidor frente a la aplicación que cachea respuestas para **todos** los clientes
  - además: terminación TLS, compresión, límite de tasa, balanceo (p. ej., NGINX)

Redes de distribución de contenido (cap. 15)



DNS/anycast lleva al PoP cercano · conexiones persistentes al origen · capas de caché

Cómo funciona una CDN

- **Enrutamiento global**: balanceo por **DNS** (y/o **anycast**) que considera ubicación, salud y carga
- **Rutas optimizadas** y **conexiones persistentes** entre PoP y origen
  - el *handshake* TCP/TLS del cliente se hace con un servidor **cercano** (RTT bajo)
  - ⇒ mejora la latencia **incluso para contenido no cacheable**
- **Caché en capas**: borde → caché intermedia (protege al origen) → origen
- Dentro de un PoP, la caché se **particiona** entre servidores (*hashing* consistente)

2. Particionar y almacenar

Particionamiento · Archivos · Datos (caps. 16, 17, 19)

Particionamiento (cap. 16)

- Dividir los datos en **particiones** (*shards*), cada una atendida por un nodo
- Se necesita un **mapa partición → nodo**
  - guardado en un servicio de coordinación (ZooKeeper, etcd)
  - consultado por un **gateway** o por el cliente para enrutar
- Costos:
  - consultas y **transacciones entre particiones** (2PC, Parte II)
  - **rebalanceo** cuando cambian los nodos o la carga
  - **puntos calientes**

Por rango o por *hash*

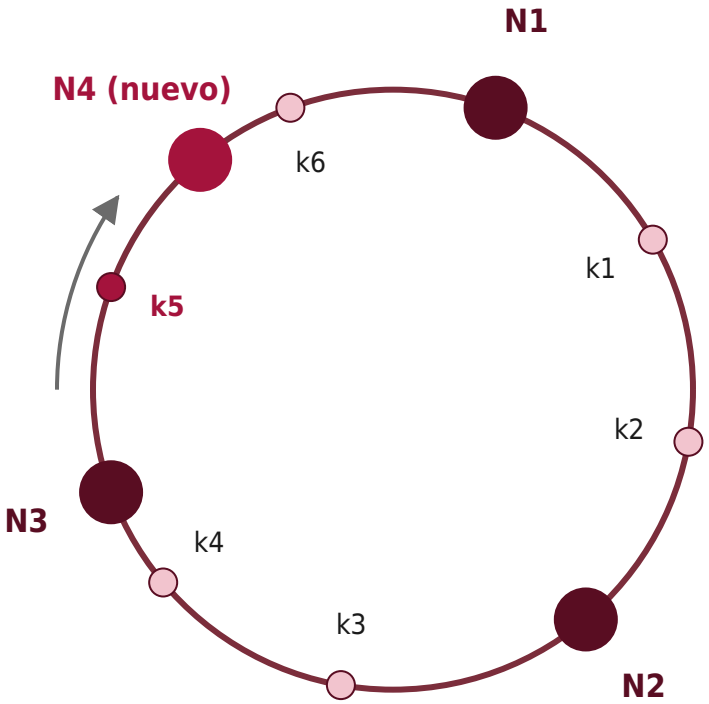


El problema de `hash(k) mod N`

Clave	hash	mod 4	mod 5
ana	1 723	3	3
luis	9 102	2	2
zoe	4 448	0	3
beto	6 731	3	1
eva	2 015	3	0

- Al pasar de 4 a 5 nodos, **la mayoría** de claves cambia de nodo → migración masiva
- En general, al pasar de N a N + 1 nodos solo ≈ 1/(N + 1) de las claves conserva su nodo

Hashing consistente



Nodos y claves se ubican en un anillo
según su hash.
Cada clave pertenece al siguiente nodo
en el sentido de las agujas del reloj.

Al agregar N4 solo se mueve k5 (de N1 a N4):
≈ 1/N de las claves, no casi todas
como con hash mod N

Rebalanceo

Particiones estáticas

- Crear desde el inicio **muchas más particiones que nodos** (p. ej., 1 000 particiones para 10 nodos)
- Al agregar un nodo, se le **mueven particiones completas**
- Simple, pero el número de particiones queda fijo

Particiones dinámicas

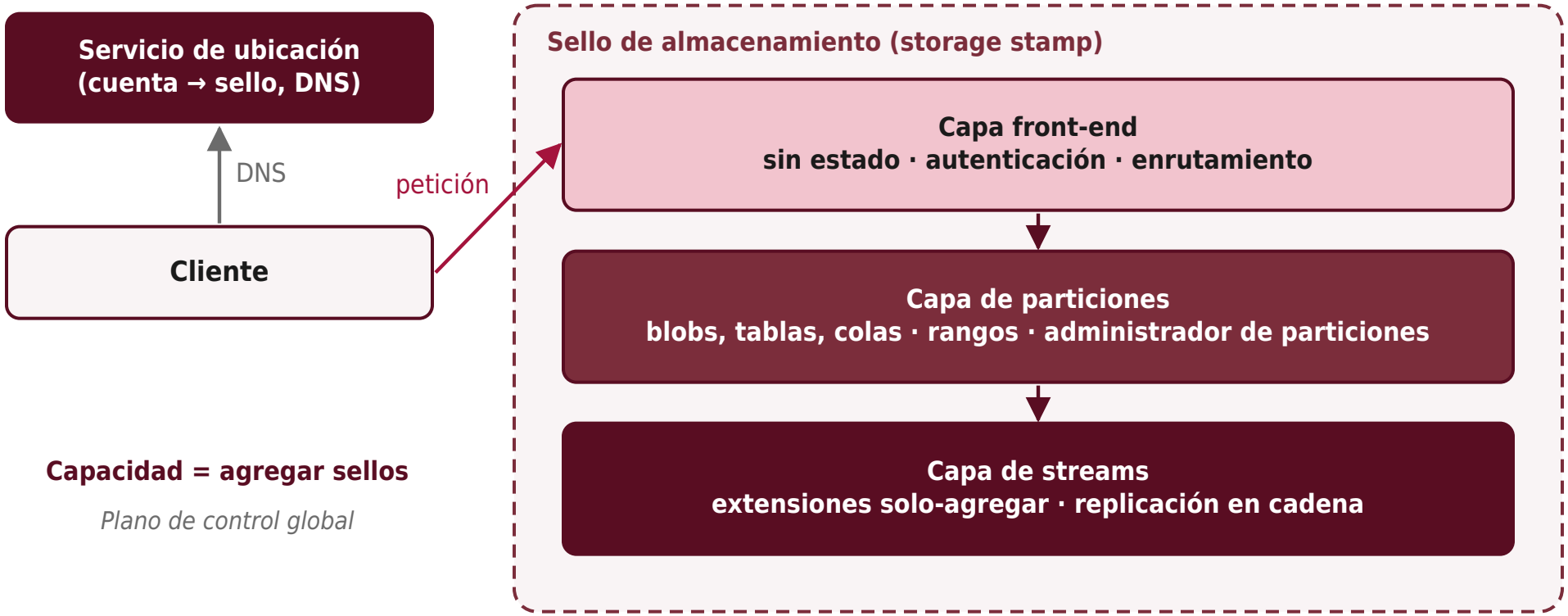
- **Dividir** una partición cuando crece demasiado; **fusionar** las pequeñas
- Se adapta a datos y carga reales (natural en particionamiento por rango)

Almacenamiento de archivos (cap. 17)

- Archivos grandes (imágenes, video, respaldos) → **almacén de objetos / *blobs*** (Amazon S3, Azure Blob Storage, Google Cloud Storage)
- No en la base de datos: esta guarda solo la **referencia** (URL o clave del objeto)
- Interfaz simple: `PUT`, `GET`, `DELETE` por clave; enorme durabilidad y escala

> Vitillo usa la arquitectura de **Azure Storage** como caso de estudio: reúne replicación en cadena, particionamiento por rango, planos de control y DNS.

Arquitectura de Azure Storage

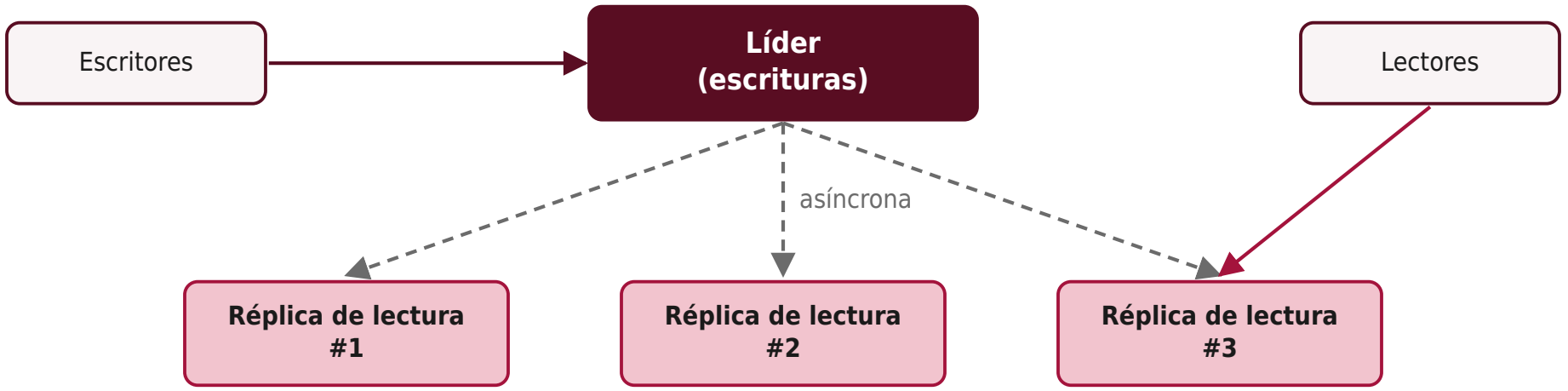


Azure Storage: conceptos reutilizados

Concepto (parte del libro)	Dónde aparece
DNS (Parte I)	El servicio de ubicación enruta cuentas a sellos
Replicación en cadena (Parte II)	Extensiones replicadas en la capa de *streams*
Plano de control con consenso (Parte II / cap. 22)	Administradores de particiones y de *streams*
Particionamiento por rango (cap. 16)	Capa de particiones; divide rangos calientes
Servicio sin estado + balanceo (cap. 18)	Capa de *front-end*
Escalar agregando unidades	Nuevos sellos

> Resultado: almacenamiento **fuertemente consistente**, disponible y escalable.

Almacenamiento de datos (cap. 19): réplicas de lectura



Escala lecturas y disponibilidad · las réplicas pueden ir atrasadas (consistencia eventual)

Relacional frente a NoSQL

	Relacional (SQL)	NoSQL
Modelo	Tablas normalizadas	Clave-valor, documentos, columnas anchas
Consultas	Flexibles: uniones, agregaciones ad hoc	Limitadas a los **patrones de acceso** diseñados
Transacciones	Generales, ACID	Limitadas (p. ej., a una partición o pocas claves)
Escalado	Tradicionalmente vertical + réplicas	**Horizontal** nativo (particiones)
Diseño	Primero el modelo, luego las consultas	**Primero las consultas**, luego el modelo

Ejemplo: DynamoDB

Tabla Pedidos · clave de partición = cliente · clave de ordenamiento = fecha

Partición 1 · hash("ana")

ana	2026-09-01	#1021
ana	2026-10-15	#1188
ana	2026-11-02	#1301

ordenado ↓

Partición 2 · hash("luis")

luis	2026-08-20	#0990
luis	2026-10-30	#1277

ordenado ↓

Consulta eficiente: pedidos de "ana" entre dos fechas (una partición, un rango)

- **Clave de partición** → por *hash* decide la partición
- **Clave de ordenamiento** → orden dentro de la partición (consultas por rango)
- Índices secundarios **locales** (misma partición) y **globales** (otra clave de partición)

Diseñar a partir de los patrones de acceso

1. Enumerar las **consultas** que la aplicación necesita
2. Elegir claves de partición y ordenamiento que las resuelvan **sin recorrer toda la tabla**
3. **Desnormalizar**: duplicar datos donde se leen; a veces **una sola tabla** con varias entidades
4. Agregar índices secundarios para los patrones restantes

> En relacional: "¿qué datos tengo?" · En NoSQL: "¿qué preguntas voy a hacer?"

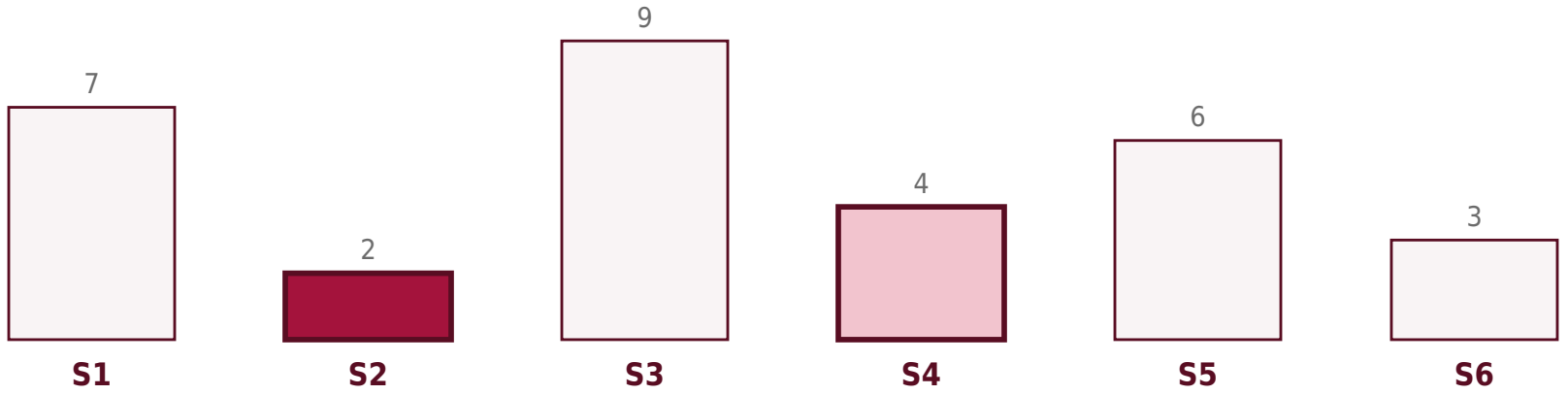
3. Repartir la carga

Balanceo de carga · Caché (caps. 18 y 20)

Balanceo de carga en red (cap. 18)

- Para escalar un servicio **sin estado**: varias réplicas detrás de un **balanceador**
- Funciones:
  - **Reparto**: *round-robin*, *hashing* consistente, menos conexiones / menor carga
  - **Descubrimiento de servicios**: registro dinámico de réplicas (etcd, Consul, DNS interno)
  - **Verificación de salud**: retirar réplicas enfermas de la rotación
- El balanceador oculta las réplicas tras **una sola dirección**

La potencia de dos opciones aleatorias



Evita que todos envíen al mismo servidor 'aparentemente libre' con información de carga desactualizada

Salud y balanceo por DNS

Verificación de salud

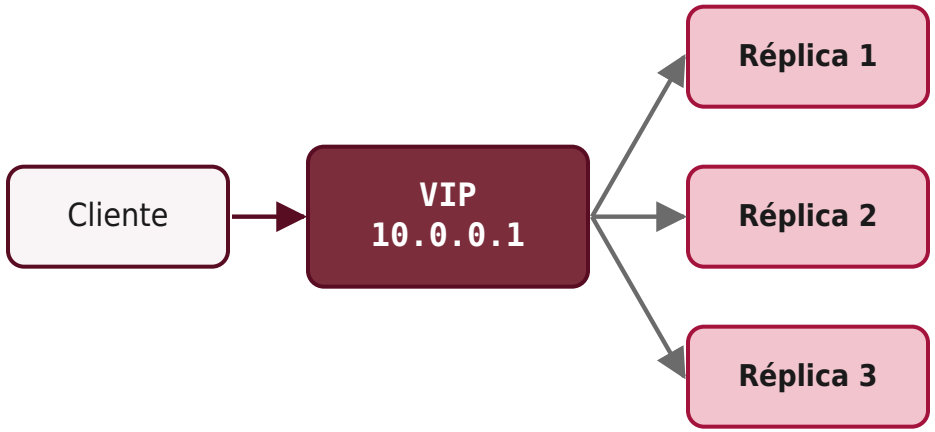
- **Pasiva**: el balanceador observa errores y *timeouts* del tráfico real
- **Activa**: consulta periódicamente un endpoint (p. ej., `GET /health`)
- Cuidado: un endpoint de salud que revisa dependencias puede **sacar a todas** las réplicas si una dependencia falla

Balanceo por DNS

- Varias IP para un nombre: simple, sin componente extra
- Pero **no conoce la salud** y las **cachés con TTL** retrasan los cambios (cap. 4)

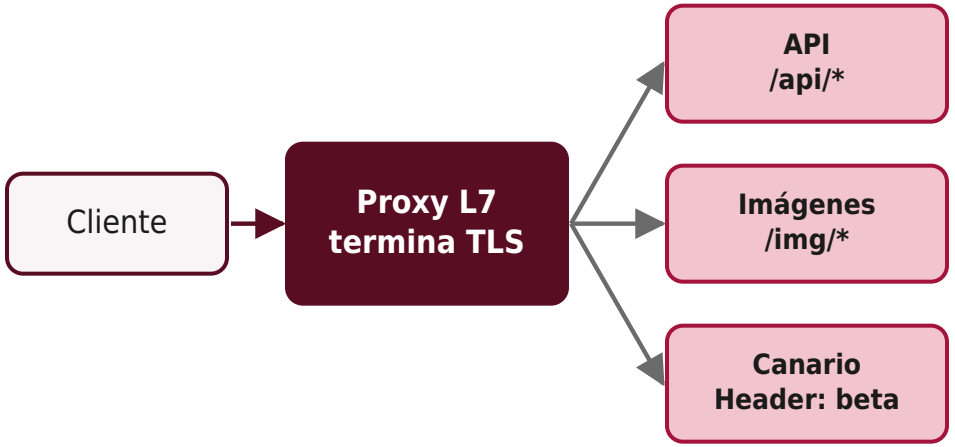
Capa 4 frente a capa 7

Capa de transporte (L4)



reparte conexiones (flujos TCP/UDP)
hash de la tupla · no ve HTTP
muy rápido · DSR posible

Capa de aplicación (L7)



reparte peticiones HTTP
enruta por ruta, encabezados, cookies
reintentos, límite de tasa · más costoso

L4, L7 y *sidecars*

	L4 (transporte)	L7 (aplicación)
Unidad	Conexión / flujo	Petición HTTP
Ve el contenido	No	Sí (ruta, encabezados, *cookies*)
TLS	Pasa cifrado	Puede **terminarlo**
Rendimiento	Muy alto	Menor (más trabajo por petición)
Funciones extra	Pocas	Reintentos, límite de tasa, despliegues canario

- L7 como ***sidecar*** junto a cada servicio → **malla de servicios** (Envoy, Istio, Linkerd)
- El balanceador también debe ser **redundante** (varias instancias, *anycast*, activo-pasivo)

Caché (cap. 20)

- Guardar resultados **costosos** de obtener para servirlos más rápido
- Métrica clave: **tasa de aciertos** (*hit ratio*) — depende de cuánto se repiten los accesos
- Políticas:
  - **Expiración** (TTL): limita cuán viejos pueden ser los datos
  - **Desalojo** al llenarse: p. ej., **LRU** (el usado hace más tiempo)

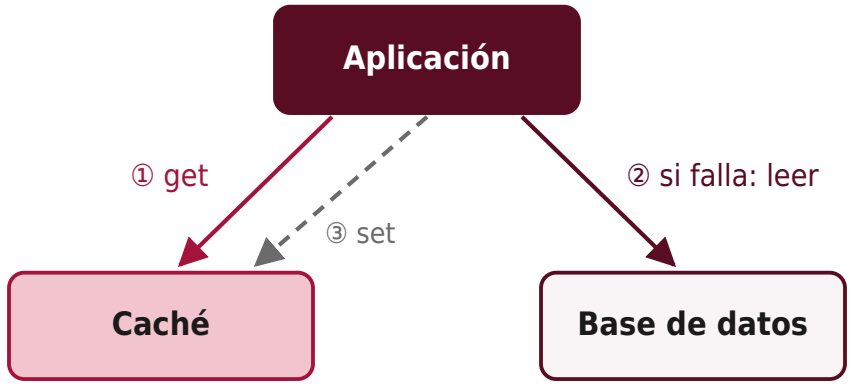
```
from functools import lru_cache
@lru_cache(maxsize=1024)          # caché local en proceso, política LRU
def precio(producto_id): return consultar_bd(producto_id)
```

Caché local frente a externa

	Local (en proceso)	Externa (Redis, Memcached)
Latencia	Mínima	Una ida y vuelta de red
Compartida entre instancias	No → duplicación e inconsistencias	Sí
Tras un despliegue	**Fría**: todas las instancias van a la BD	Se conserva
Escala	Limitada a la memoria del proceso	Particionada (*hashing* consistente) y replicada
Riesgo	Avalancha sobre la BD (*thundering herd*)	**Nueva dependencia** que puede fallar

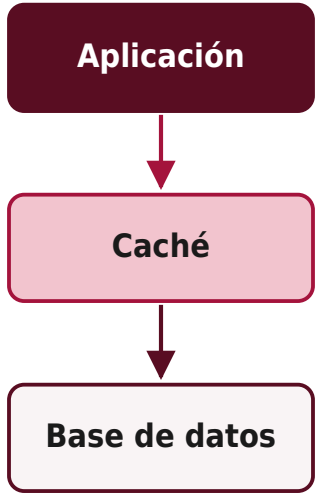
Patrones de acceso a la caché

Cache-aside (lateral)



la aplicación coordina ambas

Inline (read/write-through)



la caché
accede a la BD

La caché no es una muleta

- Si la base de datos **solo funciona gracias a la caché**, una caché vacía (reinicio, despliegue, falla) la **tumba**
- Mitigaciones:
  - dimensionar la base de datos para un nivel razonable de fallos de caché
  - **calentar** la caché gradualmente; limitar peticiones concurrentes por la misma clave
  - **descartar carga** (*load shedding*) si la base de datos se satura
- Recordar la **consistencia**: la caché puede servir datos **viejos** hasta que expiren o se invaliden

4. Descomponer

Microservicios · Planos de control y datos · Mensajería (caps. 21–23)

Microservicios (cap. 21)

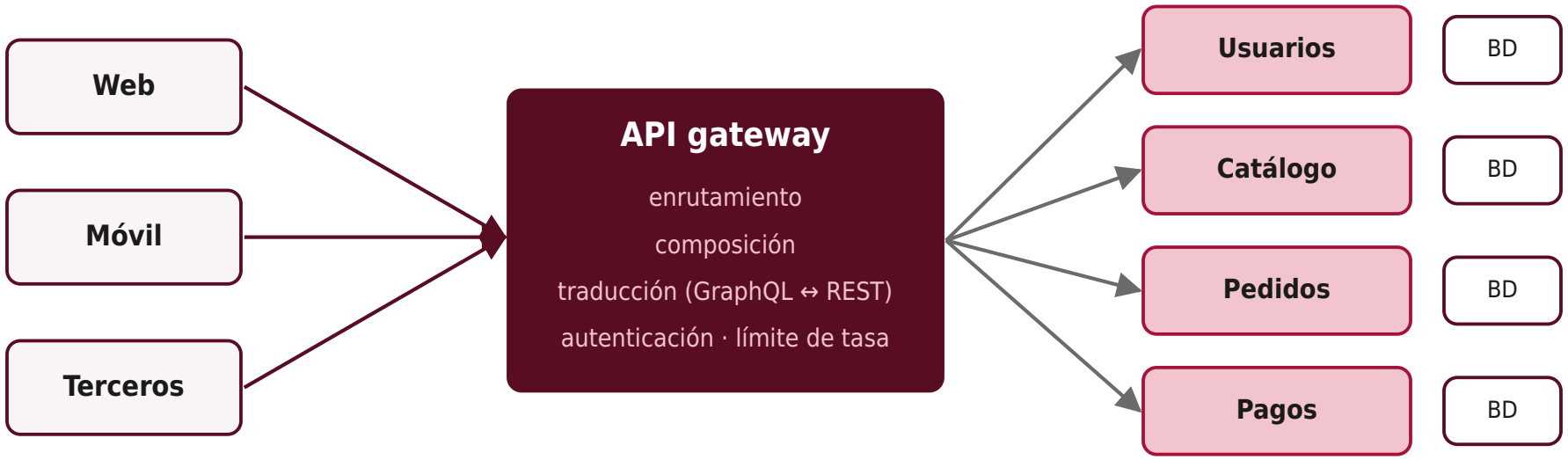
- Dividir la aplicación en **servicios independientes**, cada uno con **su propia base de datos**, comunicados por API
- Beneficios:
  - equipos que **desarrollan, despliegan y escalan** su servicio de forma independiente
  - límites claros de responsabilidad
  - escalar solo lo que lo necesita
- Muchos equipos grandes los adoptan cuando el monolito frena el desarrollo

Los costos de los microservicios

- **Proliferación tecnológica**: cada equipo con su lenguaje y sus herramientas
- **Comunicación**: llamadas por red → latencia, fallas parciales, versionado de API
- **Acoplamiento**: límites mal trazados → cambios que tocan varios servicios ("monolito distribuido")
- **Aprovisionamiento**, **pruebas** de extremo a extremo y **operación** más difíciles
- **Consistencia eventual**: sin transacciones compartidas → sagas y *outbox* (Parte II)

> Recomendación del libro: **empezar con un monolito** modular y separar cuando los límites estén claros.

API *gateway*



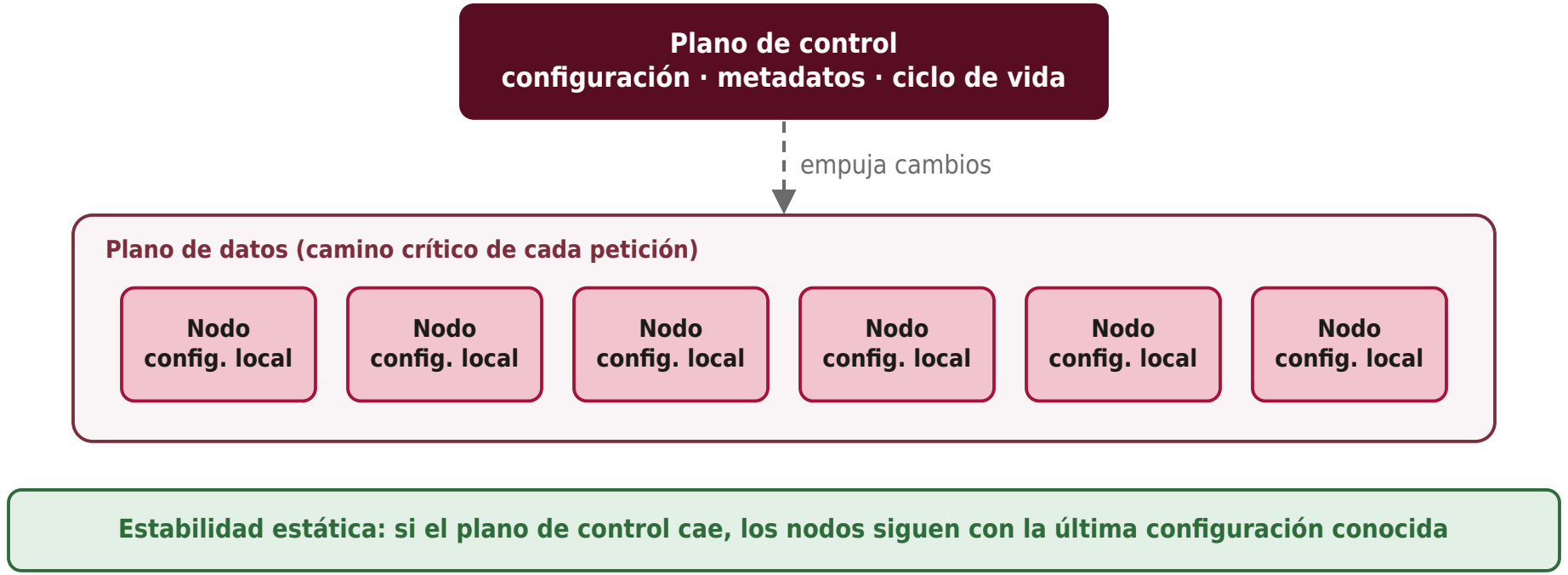
Un punto de entrada · cada servicio con su propia base de datos

Funciones del *gateway*

- **Enrutamiento**: `/pedidos/*` → servicio de pedidos
- **Composición**: una petición del cliente → varias internas → una respuesta combinada
- **Traducción**: GraphQL ↔ REST, HTTP ↔ gRPC; versiones de API
- **Funciones transversales**: autenticación y autorización (*tokens* JWT), límite de tasa, caché, registro

Riesgos: cuello de botella, punto único de falla, y si acumula lógica de negocio, **acopla** a todos los equipos

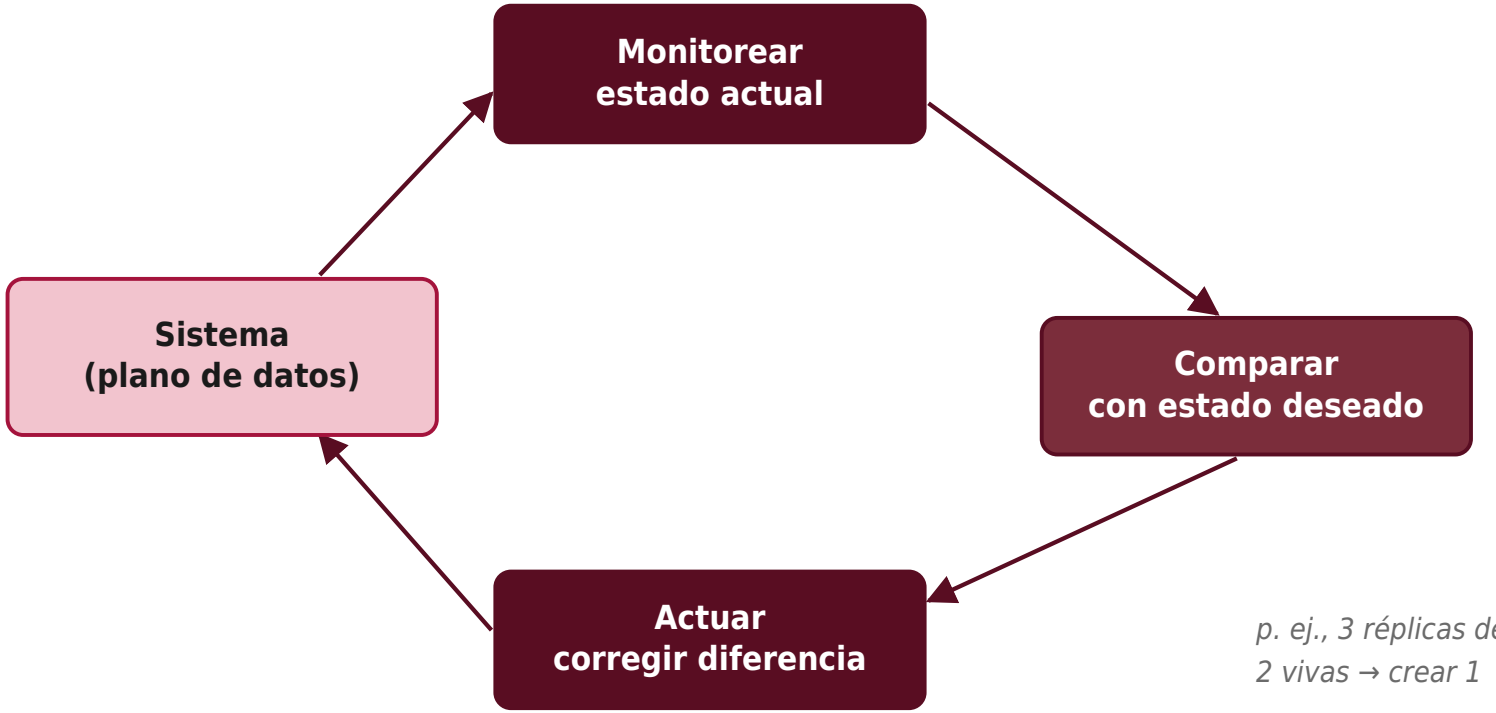
Planos de control y planos de datos (cap. 22)



Desequilibrio de escala y estabilidad estática

- El plano de datos suele ser **mucho más grande** que el de control
- Si **todos** los nodos de datos piden configuración a la vez (p. ej., tras un reinicio masivo), **saturan** el plano de control
  - mitigaciones: que el control **empuje** cambios, cachés de configuración, límites de tasa, espaciar reintentos
- **Estabilidad estática**: si el plano de control **cae**, el plano de datos **sigue operando** con la última configuración conocida
  - se pierde la capacidad de **cambiar**, no la de **servir**

El plano de control como lazo de control



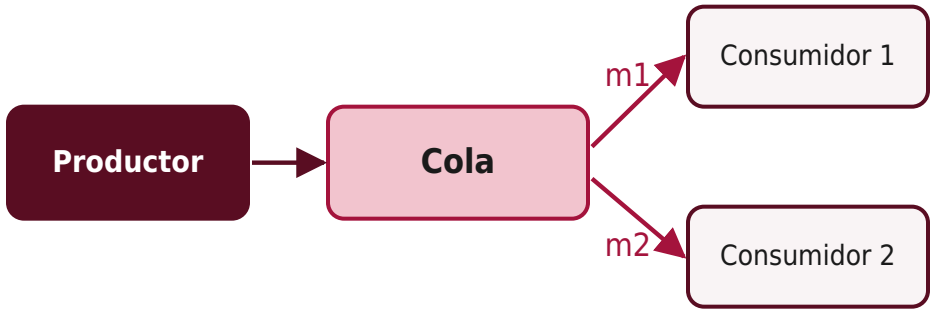
p. ej., 3 réplicas deseadas,
2 vivas → crear 1

Mensajería (cap. 23)

- Comunicación **asíncrona** a través de un **canal** (cola o *broker*): SQS, RabbitMQ, Kafka
- Beneficios:
  - **desacopla** emisor y receptor (no necesitan estar disponibles al mismo tiempo)
  - **nivela la carga**: la cola absorbe los picos
  - permite procesar **por lotes** y escalar consumidores
- Patrones: **unidireccional**, **petición-respuesta** (dos canales + id de correlación), **difusión**

Tipos de canal

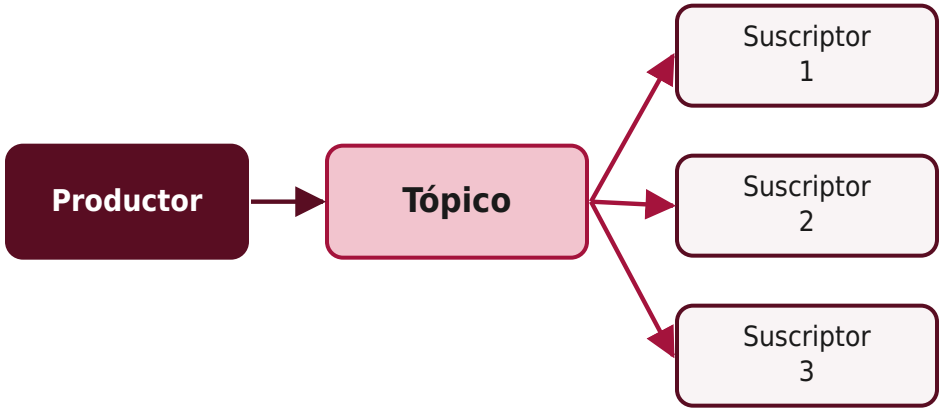
Punto a punto



cada mensaje → un solo consumidor

reparto de trabajo · nivelación de carga

Publicación-suscripción



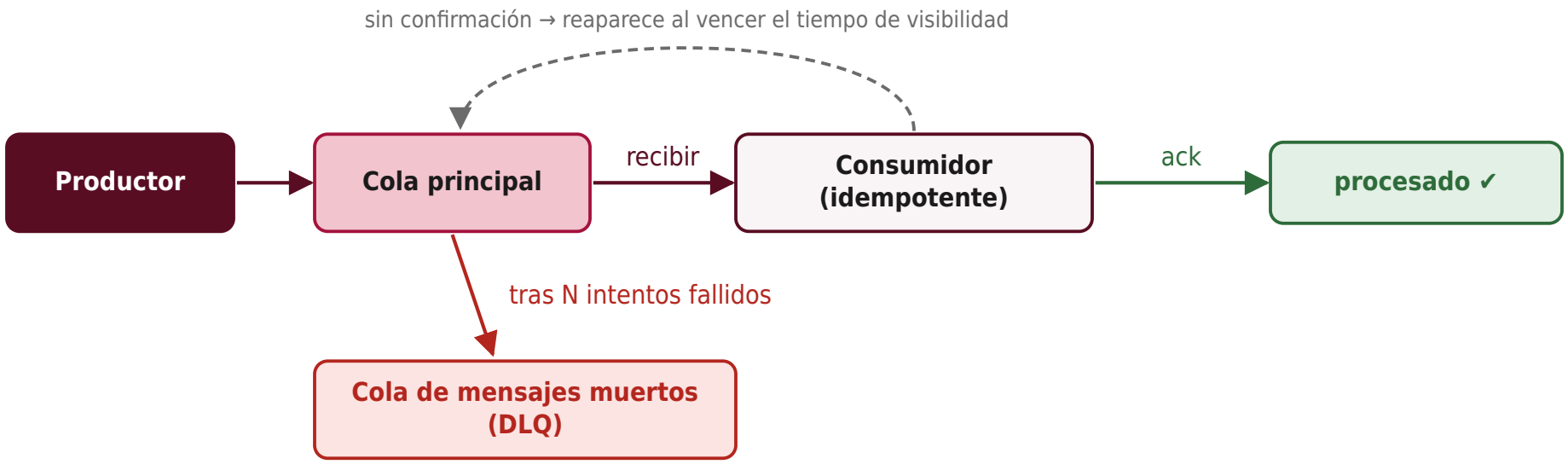
cada mensaje → todos los suscriptores

difusión de eventos

Garantías: ¿exactamente una vez?

- Las garantías **varían** entre implementaciones: orden, entrega, durabilidad
- **Exactamente una vez** no es posible en general (Parte II: no se distingue un consumidor lento de uno caído)
- Lo habitual: **al menos una vez**
  - el consumidor **confirma** (*ack*) tras procesar
  - si no confirma antes del **tiempo de visibilidad**, el mensaje **reaparece**
- ⇒ **efectivamente una vez** = al menos una vez + consumidor **idempotente** (o deduplicación por id)

Fallas y acumulación



Al menos una vez + consumidor idempotente = efectivamente una vez · mensajes envenenados → DLQ

- **Acumulación** (*backlog*): si producen más rápido de lo que se consume, la cola crece → **monitorear** tamaño y antigüedad del mensaje más viejo; escalar consumidores

Ideas clave de la Parte III

1. Tres patrones: **descomposición funcional**, **particionamiento**, **replicación**
2. **Caché HTTP** (`Cache-Control`, `ETag`) y **CDN** acercan el contenido y reducen RTT
3. **Particionar** por rango (rangos, puntos calientes) o por *hash* (uniforme); ***hashing* consistente** para rebalancear
4. **Azure Storage** combina DNS, particiones por rango y replicación en cadena
5. **Balanceo** L4/L7, salud y descubrimiento; **potencia de dos opciones**
6. **Caché** local o externa; nunca como muleta
7. **Microservicios** con costos; **gateway**; **planos** de control y datos con estabilidad estática
8. **Mensajería**: al menos una vez + idempotencia; **DLQ** para mensajes envenenados

Lecturas y referencias

- R. Vitillo, *Understanding Distributed Systems*, 2.ª ed., caps. 14–23
- RFC 9111 — *HTTP Caching*
- D. Karger *et al.*, "Consistent Hashing and Random Trees", 1997
- B. Calder *et al.*, "Windows Azure Storage: A Highly Available Cloud Storage Service with Strong Consistency", SOSP 2011
- M. Mitzenmacher, "The Power of Two Choices in Randomized Load Balancing", 2001
- Documentación de Amazon DynamoDB: *Best practices for designing and architecting*

Siguiente: Parte IV — Resiliencia

¿Preguntas?

Parte III · Escalabilidad

Sistemas Distribuidos
Universidad Industrial de Santander